

DESIGN & CALCULATIONS

Truth Table:

For a single adder block:

Ai Bi Ci	Gi	Pi	(Pi*Ci)	C+	Si
000	0	0	0	0	0
001	0	0	0	0	1
010	0	1	0	0	1
011	0	1	1	1	0
100	0	1	0	0	1
101	0	1	1	1	0
110	1	0	0	1	0
111	1	0	0	1	1

Logic Equations:

Unlike the Ripple Carry Adder (RCA), the Carry Look Ahead Adder (CLA) calculates carry bits for all stages simultaneously rather than sequentially, making it significantly faster. Instead of waiting for the previous carry, the CLA uses:

- Generate ($G_i = A_i * B_i$): where a carry is generated by the A and B at the position
- Propagate ($P_i = A_i \wedge B_i$): where we see if AB has odd parity that will allow a carry to be propagated (carried over from the previous digit)
- Sum ($S_i = P_i \wedge C_i$): the final sum/ output

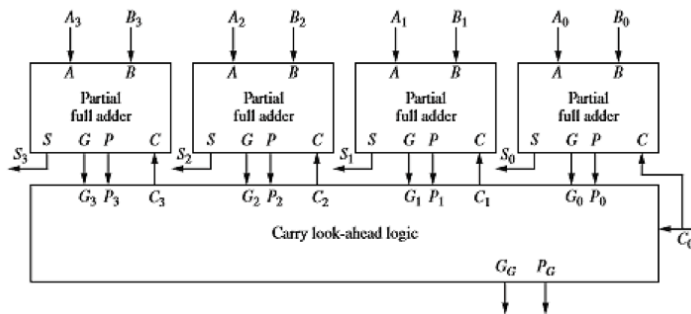
The carry formula works out to: $C_{i+1} = G_i + (P_i * C_i)$, making the function recursive, as each C_i is replaced with the formula for the previous carry. With the CLA, the final output doesn't need to wait for each full-adder to finish calculating in sequence; only the first full-adder needs to finish its calculations for G_0 and P_0 , before the rest of the adder blocks can use those outputs to add in parallel. With $C_0 = C_{in}$ being the additional carry in if we're simulating a carry flag from a previous operation, the equations for the C bus can be expanded to:

- $C_1 = G_0 + P_0 * C_{in}$
- $C_2 = G_1 + P_1 * (G_0 + P_0 * C_{in}) = G_1 + P_1 * G_0 + P_1 * P_0 * C_{in}$

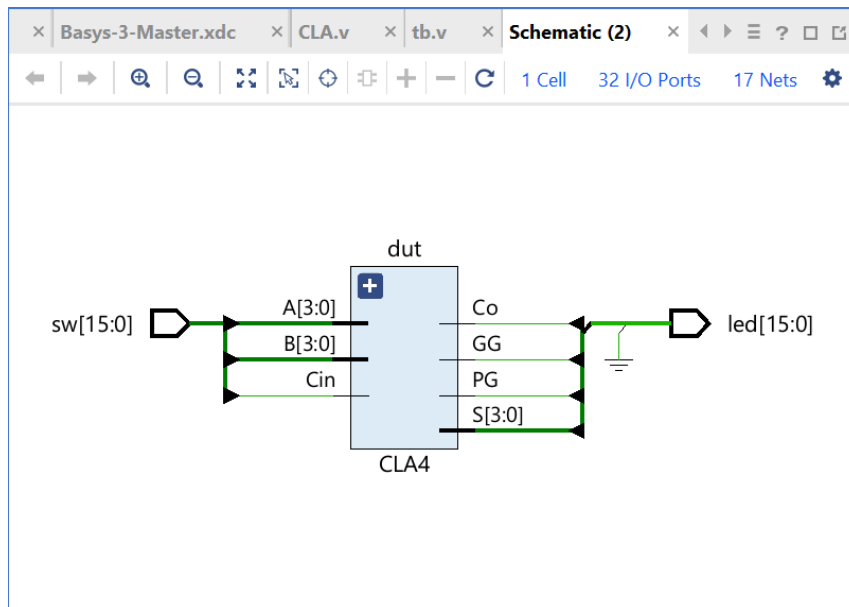
- $C_3 = G_2 + P_2 * G_1 + P_2 * P_1 * G_0 + P_2 * P_1 * P_0 * C_{in}$
- $C_4 = G_3 + P_3 * G_2 + P_3 * P_2 * G_1 + P_3 * P_2 * P_1 * G_0 + P_3 * P_2 * P_1 * P_0 * C_{in}$

To get the final sum (S) output, we just XOR all the bits of the C bus with those in the P bus. The total sum of the CLA is the 4 bits of the sum bus plus the MSB of the C bus, or the last carry.

Circuit/ Block Diagrams:



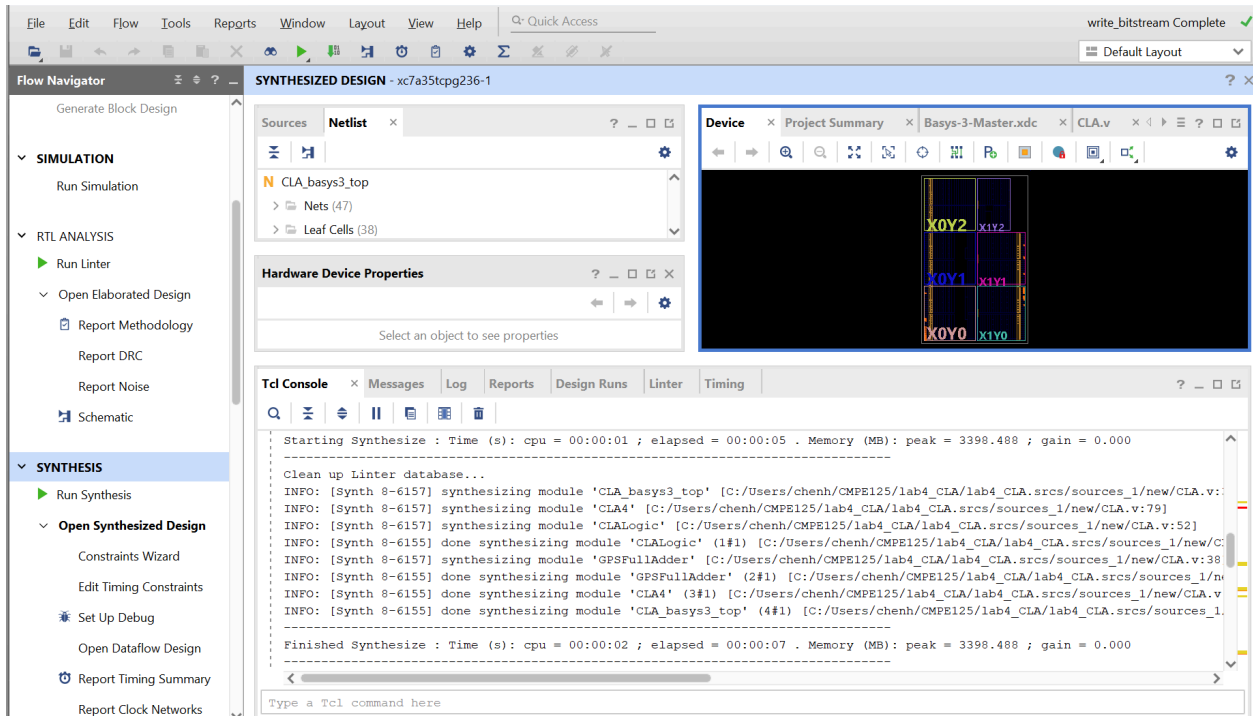
Lab Manual Block Diagram for CLA



Generated Schematic

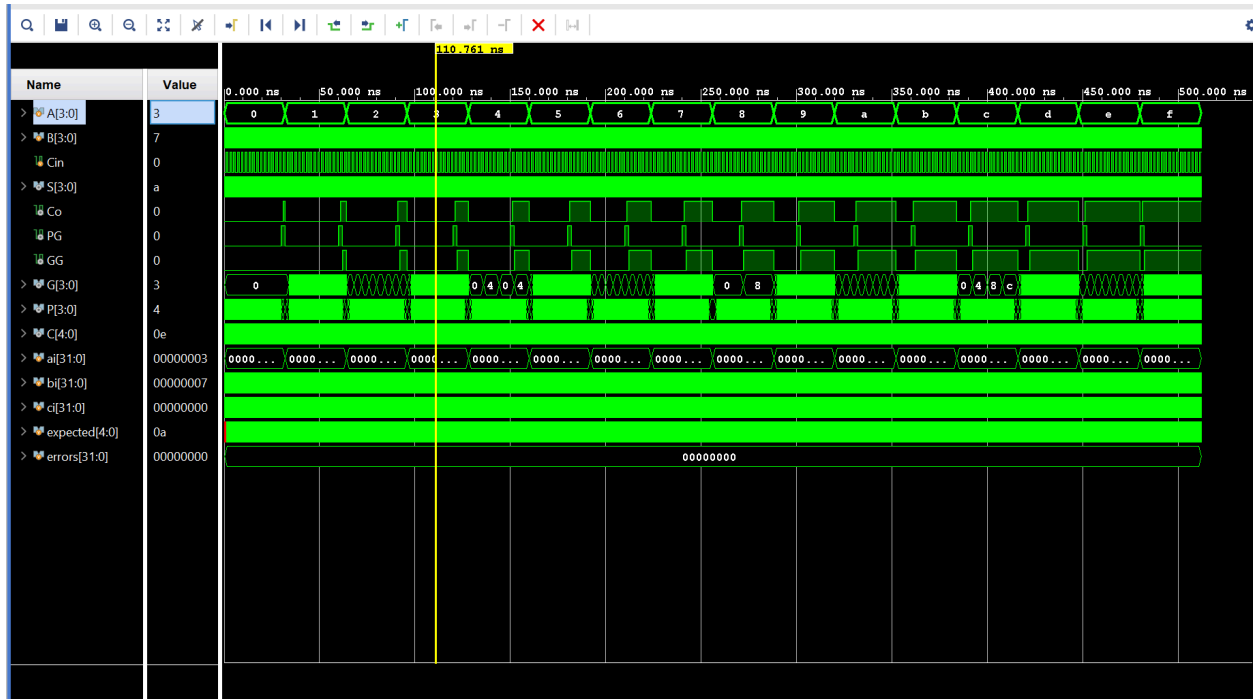
SCREENSHOTS

Compilation:



Successful Synthesis shown in Console & write_bitstream completion on upper right corner

Simulation Waveforms:





SUMMARY

Paragraph describing verification of implemented logic with the simulation results.

On the FPGA board, A and B were assigned to 4 switches each, while C[4] and S were assigned to LED lights:

- [3:0]A = sw[15]... sw[12]
- [3:0]B = sw[11]... sw[8]

- $C_{in} = sw[7]$
- $[4]C = led[4]$
- $[3:0]S = led[3]...led[0] \rightarrow$ total 5-bit sum is in $led[4:0] = \{C_4, S_3, S_2, S_1, S_0\}$

The final propagate ($PG=P_3*P_2*P_1*P_0$) and the final generate ($GG=G_3+P_3*G_2+P_3*P_2*G_1+P_3*P_2*P_1*G_0$) will be displayed in leds:

- $PG = led[15]$
- $GG = led[14]$

Simulation results confirmed that the CLA correctly computed propagate (P), generate (G), carry (C), and sum (S) signals according to the derived logic equations. For each applied test vector, the waveform verified that all the variables, including internal registers, matched their expected values and that the carry injections occurred properly at each stage. Observed LED outputs matched the expected simulation results for cases where carries were generated internally as well as propagated through multiple stages. This confirms that the CLA computes carries in parallel instead of rippling and that the Verilog implementation behaves as designed.

VERILOG CODE

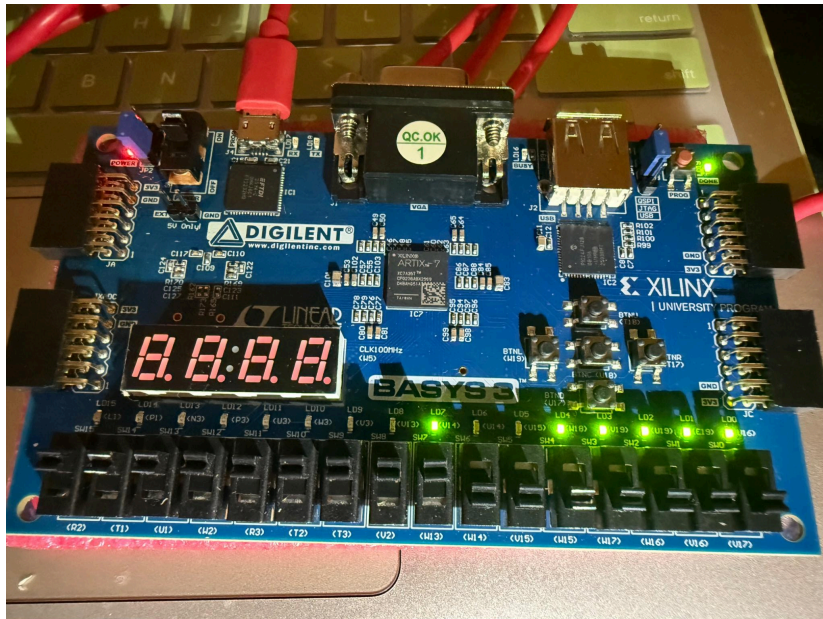
See attached files.

TROUBLESHOOTING

It was a bit tricky writing the testbench and figuring out how to verify the expected outputs in the S-bus were what they were supposed to be, as well as how the syntax for it worked. Since the waveform shows “real time” which includes the short moments after the wait “#1” when inputs change, the checker had to compare the values after waiting for the logic to settle. Times when S would not match the expected included right at time $t=0$ and immediately after each stimulus change (the window from the instant new inputs are assigned at $t=T$ until the sampling time at $t=T+1ns$). Learning to use the strict inequality operator ($!=$) was also important since it correctly detects unknown X values during simulation.

In addition, this was the first time during labs that multiple modules were placed in the module file and were used as helper modules to each other. This introduced new concepts about hierarchical design and signal flow, like how the GPSFullAdder modules generate the bitwise G and P signals, which are then fed into the CLALogic module to compute all carries simultaneously. At first, there was confusion about whether modules execute in a certain order, but Verilog describes hardware structure and not sequential software execution, so it doesn't matter what order the modules are in inside the file.

FPGA BOARD OUTPUT

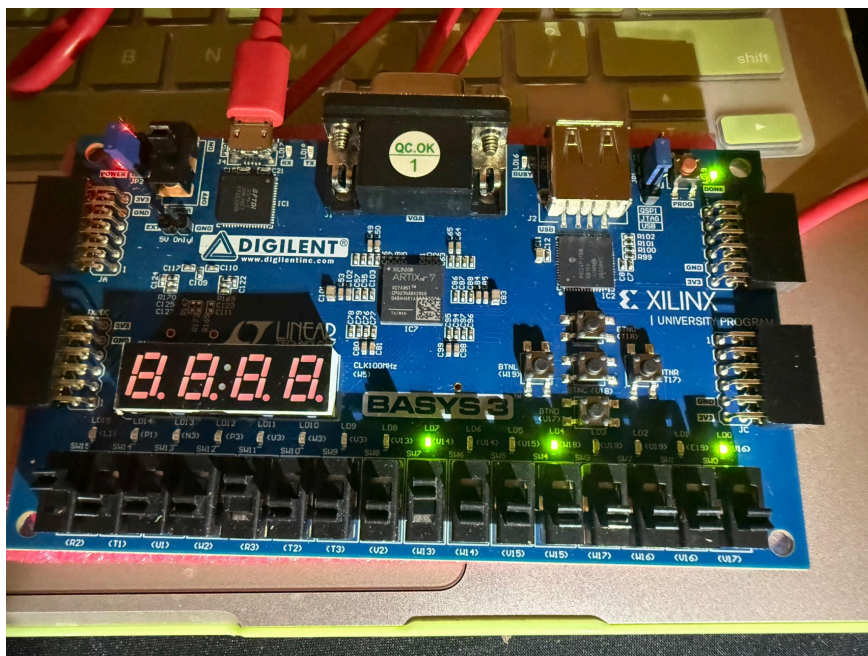


$$A = 4'b1111 = 15$$

$$B = 4'b1111 = 15$$

$$C_{in} = 1'b1 = 1$$

$$S = 5'b11111 = 15+15+1 = 31$$

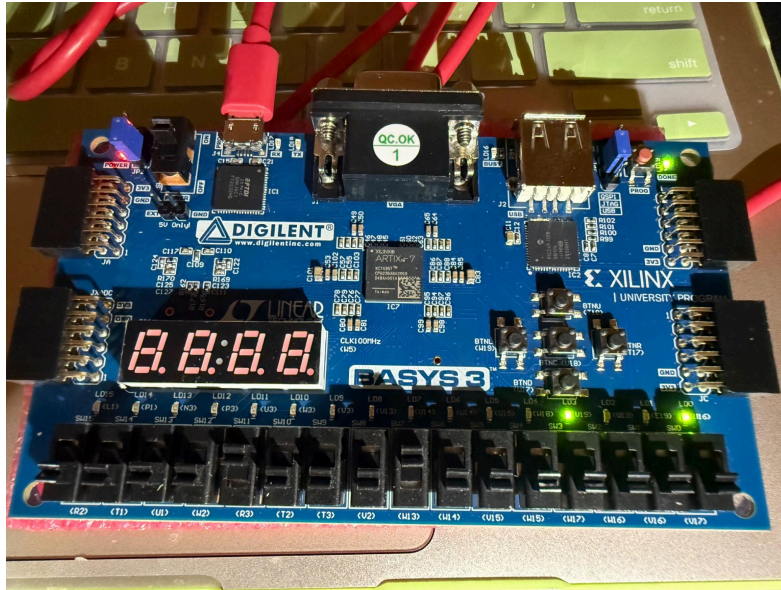


$$A = 4'b1000 = 8$$

$$B = 4'b1000 = 8$$

$$C_{in} = 1'b1 = 1$$

$$S = 5'b10001 = 8+8+1 = 17$$



$$A = 4'b0000 = 0$$

$$B = 4'b1000 = 8$$

$$C_{in} = 1'b1 = 1$$

$$S = 5'b10001 = 0+8+1 = 9$$